

Document de justifications

Choix organisationnels, techniques et architecturaux

Choix techniques

Le langage serveur est **JavaScript**, comme imposé par le sujet. Pour le framework backend, on a choisi **Express**. C'est un framework minimaliste qui ne nous impose pas de structure rigide, ce qui nous a laissé libres d'organiser le code comme on le souhaitait. On l'avait déjà utilisé dans des projets précédents, donc on était à l'aise dessus.

On n'a pas utilisé **TypeScript**, tout simplement parce qu'on est moins à l'aise avec. Sur un projet de cette taille, avec des délais contraints, on a préféré rester sur ce qu'on maîtrise.

Côté client, on a choisi **React** pour les mêmes raisons : c'est le framework que le groupe connaît le mieux. Couplé à **Vite** pour le bundling, le setup est rapide et le hot reload en développement est idéal. Pour les composants UI, on a utilisé **shadcn/ui** basé sur Radix UI, avec **Tailwind CSS** pour le style. Ça nous a évité de créer des composants accessibles from scratch (dialogs, dropdowns, etc.) tout en gardant la main sur le rendu final.

Pour la base de données, on a choisi **PostgreSQL**. C'est une base relationnelle très complète, bien plus performante et mature que ses concurrents open source sur des charges importantes, et surtout elle gère un large éventail de types de données. Elle est aussi très bien supportée par l'écosystème Node.js. Pour l'accès aux données, on a utilisé **Prisma** car plusieurs membres du groupe l'avaient déjà utilisé : la définition du schéma est claire, la génération du client typé facilite les requêtes, et Prisma Studio est pratique pour inspecter la base en développement.

Pour les **communications en temps réel**, on a fait le choix d'utiliser l'API WebSocket native (**ws** côté serveur, WebSocket natif côté navigateur) sans passer par une bibliothèque comme Socket.IO. L'objectif était de comprendre le fonctionnement du protocole de zéro, sans surcouche abstraite. Ça nous a forcés à implémenter nous-mêmes la gestion des rooms, le broadcast et la reconnexion.

Les **appels audio** sont implémentés avec **WebRTC**, l'API native du navigateur pour la communication peer-to-peer. Le serveur WebSocket sert de serveur de signaling : il relaie les offres SDP, les réponses et les candidats ICE entre les participants. On utilise un serveur **STUN public de Cloudflare** pour la résolution des adresses ICE. On n'a pas mis en place de serveur TURN, les appels peuvent donc échouer sur certains réseaux très restrictifs, mais c'était acceptable pour ce projet.

Choix architecturaux

L'application est divisée en **trois services distincts** : une API REST (services/api), un serveur WebSocket (services/ws) et le client React (services/client). En production, l'API REST tourne sur le **premier serveur** et le serveur WebSocket sur le **deuxième serveur**, tandis que le client React est compilé en fichiers statiques et peut être servi depuis l'un ou l'autre, ou un CDN dédié. En développement, tout tourne sur la même machine avec trois ports distincts (3001, 3002, 5173) grâce à Foreman qui lit le Procfile.

On a choisi de séparer l'API REST du serveur WebSocket plutôt que de les fusionner parce que les responsabilités sont très différentes. L'API gère les opérations CRUD classiques avec authentification, validation et accès à la base de données. Le serveur WS gère de la communication en temps réel (changements de document, signaling WebRTC, messages de chat) et maintient un état en mémoire (rooms, appels actifs). Les mélanger aurait rendu le code plus difficile à maintenir et à faire évoluer indépendamment.

Ce découplage a aussi un avantage pour la **scalabilité** : les connexions WebSocket sont persistantes et consomment des ressources de manière continue, contrairement aux requêtes HTTP. En les isolant sur un serveur dédié, on peut faire évoluer les deux indépendamment.

La communication entre les services côté client est directe : le navigateur parle à l'API via HTTP et au serveur WS via WebSocket. Il n'y a pas de communication serveur-à-serveur : si une action côté API doit notifier des clients en temps réel, c'est le client qui, après avoir reçu la réponse HTTP, envoie un message au WS pour broadcaster. Ce choix simplifie l'architecture en évitant un bus de messages sur plusieurs services.

La base de données est accessible uniquement depuis l'API. Le serveur WebSocket ne fait aucune requête SQL : pour authentifier une connexion entrante, il délègue la vérification du token à l'API via un appel HTTP interne protégé par un secret partagé (**WS_INTERNAL_SECRET**). Ce découplage maintient le service WS léger et concentré sur son rôle de relai de messages.

Pourquoi ce choix plutôt que les alternatives

Deux autres approches étaient envisageables :

- **Connecter le service WS directement à la base de données.** C'est la solution la plus simple, mais elle contredit le principe d'architecture qu'on s'est fixé. Si le WS accède aussi à Postgres, on se retrouve avec deux services partageant le même schéma, les mêmes credentials et les mêmes règles de validation : deux connection pools à gérer, deux services à mettre à jour à chaque migration, et une surface d'attaque plus large.
- **Faire notifier le service WS par l'API lors d'un bannissement.** L'idée serait que quand un admin bloque un utilisateur, l'API envoie un message au WS pour fermer la connexion active immédiatement. Le problème : on introduit une **dépendance bidirectionnelle**. En production avec plusieurs instances WS, l'API devrait notifier toutes les instances, ce qui nécessite un bus de messages (Redis pub/sub). Cette approche est pertinente pour des systèmes exigeant une révocation instantanée, mais pour notre cas d'usage, le délai acceptable est celui de la durée de vie du JWT.
- **L'appel HTTP interne au moment de la connexion** est le bon compromis : la vérification est fraîche, la logique d'authentification reste dans un seul service, et le WS ne connaît pas la base de données. En production, l'endpoint est protégé par le secret partagé dans le header **X-Internal-Secret** et par l'isolation réseau. La latence est négligeable car l'appel n'a lieu qu'à l'établissement de la connexion WebSocket, pas à chaque message.

Choix organisationnels

Le projet a été découpé en grandes fonctionnalités développées de manière itérative. On a commencé par le socle commun : authentification, gestion des documents et arborescence de fichiers. Une fois cette base stable, on a ajouté les fonctionnalités temps réel (édition collaborative, appels audio) puis les fonctionnalités avancées (2FA, gestion des permissions, chat).

On a utilisé **Git** avec des branches par fonctionnalité et des pull requests pour la revue de code. Ça nous a permis de travailler en parallèle sans trop se bloquer mutuellement. La gestion des tâches s'est faite de façon informelle au début, puis on a adopté un tableau simple pour suivre ce qui était en cours et ce qui restait à faire. Les décisions techniques importantes ont été prises collectivement.

Hierarchie des rôles et rôle SUPERADMIN

On a mis en place trois niveaux de rôles : **USER**, **ADMIN** et **SUPERADMIN**. Au départ on pensait s'en sortir avec deux niveaux, mais on a rapidement vu qu'un seul rôle d'admin posait un problème : n'importe quel admin aurait pu se promouvoir lui-même ou donner/enlever des droits d'admin, sans aucun contrôle.

Un **ADMIN** peut gérer les utilisateurs : créer des comptes, bloquer ou débloquer des gens, consulter la liste des membres.

Le **SUPERADMIN** a deux capacités supplémentaires :

- **Changer les rôles** : seul un SUPERADMIN peut promouvoir ou rétrograder un utilisateur. Un ADMIN ne peut pas se passer lui-même SUPERADMIN, ni donner ce rôle à quelqu'un d'autre.
- **Créer des comptes SUPERADMIN** : un ADMIN peut créer des USER ou des ADMIN, mais pas des SUPERADMIN. Seul un SUPERADMIN peut en créer un autre.

Côté documents, les admins et superadmins voient tous les documents de la plateforme sans avoir besoin d'être invités, ce qui leur permet d'intervenir sur le contenu si besoin.

Sécurité et analyse statique du code

On a utilisé **SonarQube** pour faire une analyse SAST du projet. L'idée était de scanner le code à la recherche de failles potentielles sans avoir à les trouver manuellement. L'analyse a mis en évidence quelques points qu'on a corrigés dans la foulée, ce qui nous a permis d'avoir un regard extérieur sur la qualité et la sécurité du code, au-delà de ce qu'on aurait pu repérer lors des revues de PR.